

Rapport Technique

Jeu du Serpent en Python

Chaima

3 mai 2026

Table des matières

1	Introduction	3
2	Objectifs du Projet	3
3	Technologies Utilisées	3
3.1	Bibliothèques Python	3
4	Architecture du Code	3
4.1	Structure Générale	3
4.2	Variables Globales	3
5	Analyse Détaillée du Code	4
5.1	Configuration de l'Écran de Jeu	4
5.2	Création du Serpent	4
5.3	Création de la Nourriture	4
5.4	Système d’Affichage du Score	4
5.5	Fonctions de Mouvement	5
5.6	Fonction de Déplacement	5
5.7	Configuration des Contrôles	5
6	Boucle Principale du Jeu	6
6.1	Détection de Collision avec les Bordures	6
6.2	Détection de Collision avec la Nourriture	6
6.3	Mouvement du Corps du Serpent	6
6.4	Détection d’Auto-Collision	7
7	Analyse des Performances	7
7.1	Optimisations Présentes	7
8	Conclusion	7

1 Introduction

Ce rapport présente l'analyse technique d'un jeu du serpent (Snake Game) développé en Python utilisant la bibliothèque Turtle Graphics. Le jeu implémente les mécaniques classiques du serpent : déplacement, croissance lors de la consommation de nourriture, et détection de collisions.

2 Objectifs du Projet

- Créer un jeu interactif du serpent fonctionnel
- Implémenter un système de score
- Gérer les collisions avec les bordures et le corps du serpent
- Fournir une interface utilisateur intuitive avec contrôles au clavier

3 Technologies Utilisées

3.1 Bibliothèques Python

- **turtle** : Interface graphique pour le rendu du jeu
- **time** : Gestion des délais et temporisation
- **random** : Génération de positions aléatoires pour la nourriture

4 Architecture du Code

4.1 Structure Générale

Le programme est organisé en plusieurs sections principales :

1. Configuration initiale et variables globales
2. Création des objets graphiques (serpent, nourriture, affichage)
3. Définition des fonctions de mouvement
4. Configuration des contrôles clavier
5. Boucle principale du jeu

4.2 Variables Globales

Listing 1 – Variables de configuration

```
1 delay = 0.1      # Vitesse du jeu
2 score = 0        # Score actuel du joueur
3 segments = []    # Liste des segments du corps du serpent
```

5 Analyse Détaillée du Code

5.1 Configuration de l'Écran de Jeu

Listing 2 – Initialisation de l'écran

```
1 wn = turtle.Screen()
2 wn.title("Snake Game By Chaima")
3 wn.bgcolor("gray")
4 wn.setup(width=600, height=600)
5 wn.tracer(0) # D sactive les mises jour automatiques
```

Explication :

- `turtle.Screen()` crée la fenêtre de jeu
- `title()` définit le titre de la fenêtre
- `bgcolor()` définit la couleur de fond
- `setup()` configure les dimensions (600x600 pixels)
- `tracer(0)` améliore les performances en contrôlant manuellement les mises à jour

5.2 Création du Serpent

Listing 3 – Initialisation de la tête du serpent

```
1 head = turtle.Turtle()
2 head.speed(0) # Vitesse maximale
3 head.shape("circle") # Forme circulaire
4 head.color("black") # Couleur noire
5 head.penup() # vite de dessiner lors du d placement
6 head.goto(0,0) # Position initiale au centre
7 head.direction = "stop" # tat initial : arr t
```

5.3 Création de la Nourriture

Listing 4 – Initialisation de la nourriture

```
1 food = turtle.Turtle()
2 food.speed(0)
3 food.shape("circle")
4 food.color("red") # Couleur rouge pour la visibilit
5 food.penup()
6 food.goto(0,100) # Position initiale
```

5.4 Système d’Affichage du Score

Listing 5 – Configuration de l’affichage du score

```
1 pen = turtle.Turtle()
2 pen.speed(0)
3 pen.shape("square")
4 pen.color("white")
5 pen.penup()
6 pen.hideturtle() # Cache la tortue d’affichage
```

```
7 pen.goto(0,260)           # Position en haut de l' cran
8 pen.write("Score : 0", align="center", font=("Courier", 24, "normal"))
```

5.5 Fonctions de Mouvement

Listing 6 – Fonctions de contrôle directionnel

```
1 def go_up():
2     head.direction = "up"
3
4 def go_down():
5     head.direction = "down"
6
7 def go_right():
8     head.direction = "right"
9
10 def go_left():
11     head.direction = "left"
```

Principe : Ces fonctions modifient uniquement la variable de direction. Le mouvement effectif est géré par la fonction `move()`.

5.6 Fonction de Déplacement

Listing 7 – Logique de déplacement

```
1 def move():
2     if head.direction == "up":
3         y = head.ycor()
4         head.sety(y + 20)
5     if head.direction == "down":
6         y = head.ycor()
7         head.sety(y - 20)
8     if head.direction == "left":
9         x = head.xcor()
10        head.setx(x - 20)
11    if head.direction == "right":
12        x = head.xcor()
13        head.setx(x + 20)
```

Mécanisme : Le serpent se déplace par incréments de 20 pixels dans la direction spécifiée.

5.7 Configuration des Contrôles

Listing 8 – Liaison des touches du clavier

```
1 wn.listen()
2 wn.onkeypress(go_up, "Up")
3 wn.onkeypress(go_down, "Down")
4 wn.onkeypress(go_right, "Right")
5 wn.onkeypress(go_left, "Left")
```

6 Boucle Principale du Jeu

6.1 Détection de Collision avec les Bordures

Listing 9 – Gestion des collisions avec les murs

```

1 if head.xcor() > 290 or head.xcor() < -290 or head.ycor() > 290 or
   head.ycor() < -290:
2     time.sleep(1)           # Pause d'une seconde
3     head.goto(0, 0)         # Retour au centre
4     head.direction = "stop" # Arr t du mouvement
5     score = 0               # Remise z ro du score
6
7     # Nettoyage des segments
8     for segment in segments:
9         segment.goto(1000, 1000) # D placement hors cran
10    segments.clear()          # Vidage de la liste

```

6.2 Détection de Collision avec la Nourriture

Listing 10 – Gestion de la consommation de nourriture

```

1 if head.distance(food) < 20:
2     # Repositionnement alatoire de la nourriture
3     x = random.randint(-290, 290)
4     y = random.randint(-290, 290)
5     food.goto(x, y)
6
7     # Cr ation d'un nouveau segment
8     new_seg = turtle.Turtle()
9     new_seg.speed(0)
10    new_seg.shape("circle")
11    new_seg.color("purple")
12    new_seg.penup()
13    segments.append(new_seg)
14
15    # Mise jour du score
16    score += 10
17    pen.clear()
18    pen.write("Score : {}".format(score), align="center",
19              font=("Courier", 24, "normal"))

```

6.3 Mouvement du Corps du Serpent

Listing 11 – Logique de déplacement des segments

```

1 # D placement des segments en ordre inverse
2 for index in range(len(segments)-1, 0, -1):
3     x = segments[index-1].xcor()
4     y = segments[index-1].ycor()
5     segments[index].goto(x, y)
6
7 # Le premier segment suit la t te
8 if len(segments) > 0:

```

```
9     x = head.xcor()
10    y = head.ycor()
11    segments[0].goto(x, y)
```

Algorithme : Chaque segment prend la position du segment précédent, créant un effet de suivi naturel.

6.4 Détection d'Auto-Collision

Listing 12 – Vérification de collision avec le corps

```
1 for segment in segments:
2     if segment.distance(head) < 20:
3         time.sleep(1)
4         head.goto(0, 0)
5         head.direction = "stop"
6         score = 0
7
8         # R initialisation compl te
9         for segment in segments:
10             segment.goto(1000, 1000)
11         segments.clear()
```

7 Analyse des Performances

7.1 Optimisations Présentes

- `tracer(0)` : Désactivation des mises à jour automatiques
- `speed(0)` : Vitesse maximale pour les objets Turtle
- `penup()` : Évite le dessin lors des déplacements

8 Conclusion

Ce jeu du serpent présente une implémentation solide des mécaniques de base utilisant Python et Turtle Graphics. Le code est bien structuré et fonctionnel, avec une logique claire pour la gestion des collisions, du mouvement et du scoring. Les modifications apportées ont simplifié l'interface en supprimant le high score et corrigé la logique de mise à jour du score.

Le projet démontre une bonne compréhension des concepts de programmation orientée objet, de la gestion d'événements clavier, et de la programmation de jeux en temps réel.